

Sample Set - Assignment

Poorna Praneesha



Overview

Set 1

1. Problem Statement
2. Overview
3. System Architecture
4. Modules
 - a. PDF Extractor - Unstructured
 - b. Summarizer
 - c. Data Indexer - vector Db
 - d. Translator
 - e. Retriver
 - f. Generator

Set 2

1. Testing
 - a. Dataset

Set 3

1. FastAPI
2. Streamlit
3. Dockerisation

Problem Statement

“RAG QA Bot on Financial Report”

Assignment Overview

1. RAG Model Development
 - a. Data Parsing
 - b. Data Preprocess tables from PDF into structured formats
 - c. Indexer
 - d. Retrieval
 - e. Generator
 - f. Testing
2. Interactive QA Bot Interface
 - a. Streamlit
 - b. Dockerized application

Assignment Overview

Technical

1. RAG Model Development
 - a. Data Parsing
 - b. Data Preprocess tables from PDF
 - c. Indexer
 - d. Retrieval
 - e. Generator
 - f. Testing
2. Interactive QA Bot Interface
 - a. Streamlit
 - b. Dockerized application

Material

1. Code Implementation
2. Documentation

Assignment Overview

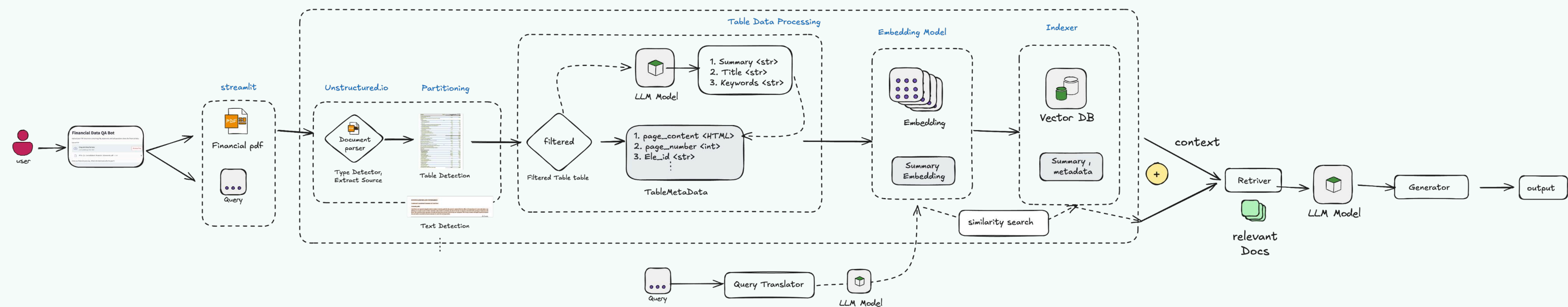
Technical

1. RAG Model Development
 - a. Data Parsing
 - b. Data Preprocess tables from PDF
 - c. Indexer
 - d. Retrieval
 - e. Generator
 - f. Testing
2. Interactive QA Bot Interface
 - a. Streamlit
 - b. Dockerized application

Material

1. Code Implementation
2. Documentation

Overall Architecture



Modules Built

1. Document Parsing
 - a. Unstructured.io
2. Data Preprocess tables from PDF
3. Indexer
 - a. ChromDB
4. Retrieval
5. Generator
6. Testing
7. Streamlit
8. Dockerized application

2. Data Preprocessing

Structured format data is preprocessed .

1. Preprocess the Table Data:

- Load the table data from a structured document.

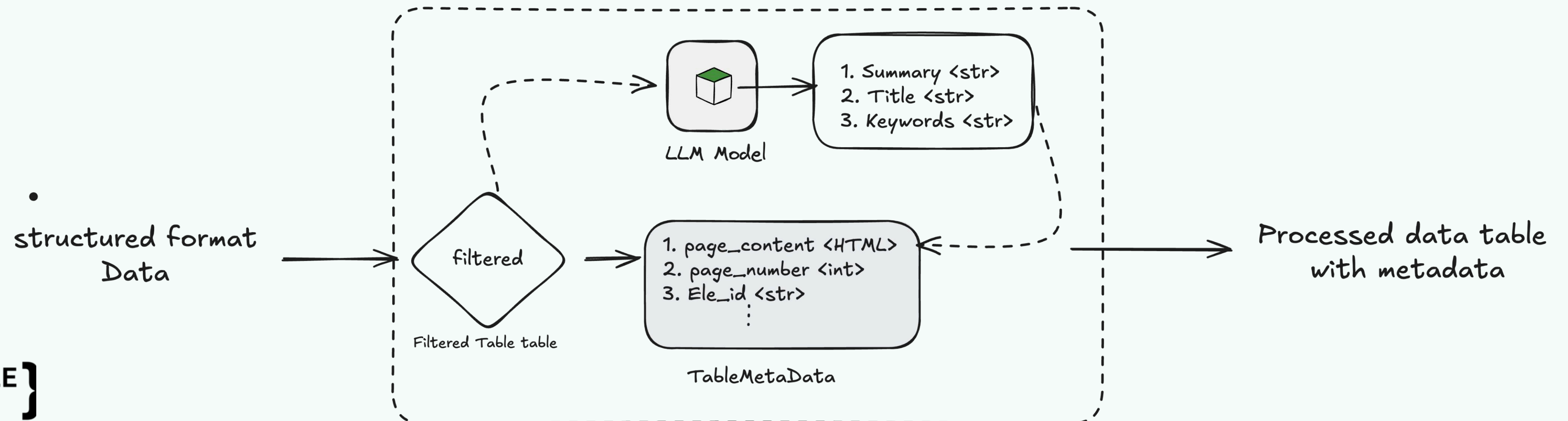
2. Generate Insights:

- Summary: Create a brief summary using a summarization model.
- Title: Generate a title based on the filtered data.
- Keywords: Extract key terms from the data.

3. Append Metadata:

- Combine the filtered table with the generated summary, title, and keywords in a structured output.

Table Data Processing

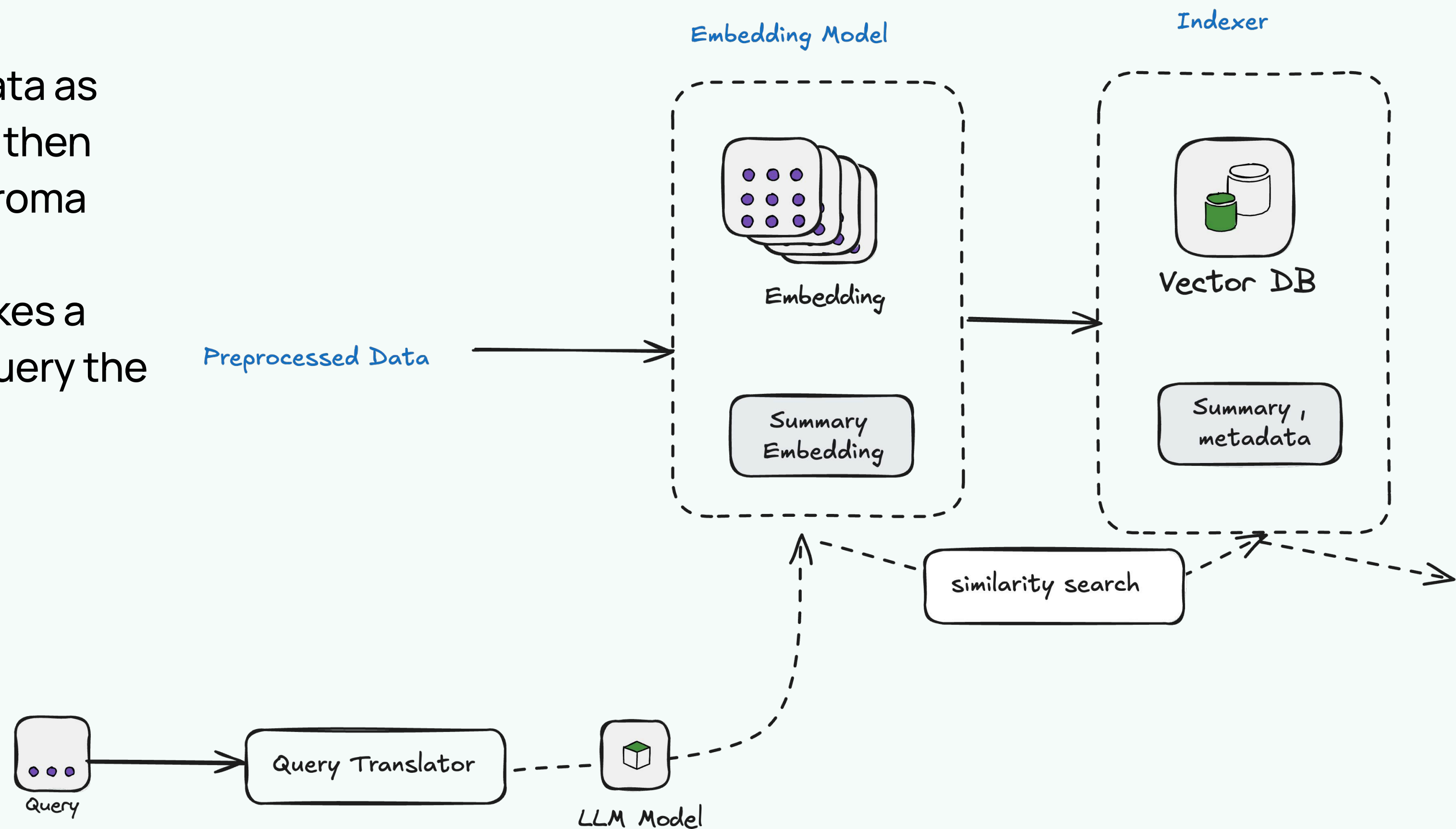


3. Indexer

Representing textual or other forms of data as vectors (numerical representations) and then indexing them into a vector database Chroma

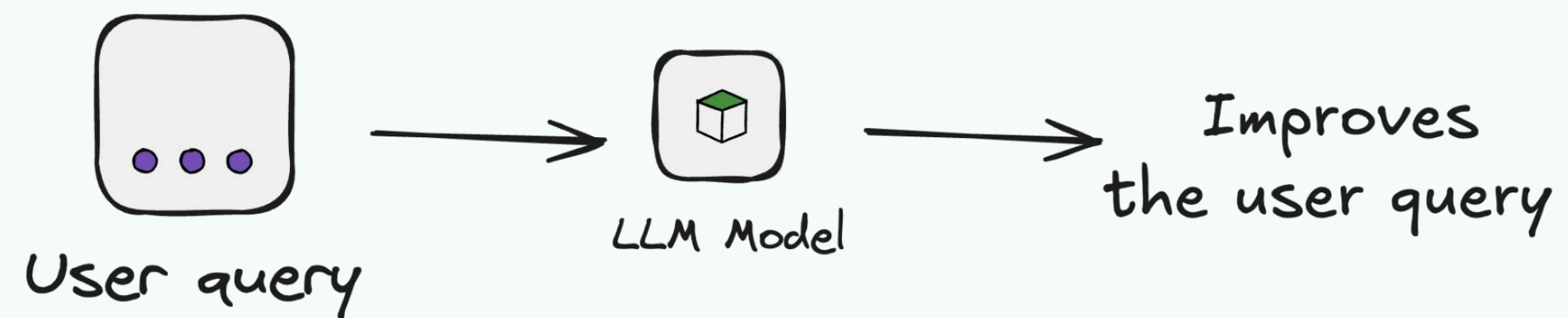
This module takes an user query, and makes a decision what type (Vector DB, Tool) of query the user requested,

- The current classification include,
- 1. infosys finanacial pdf documents
 - 2. Upload pdf document



4. Query Translator

Simple query Enhancer

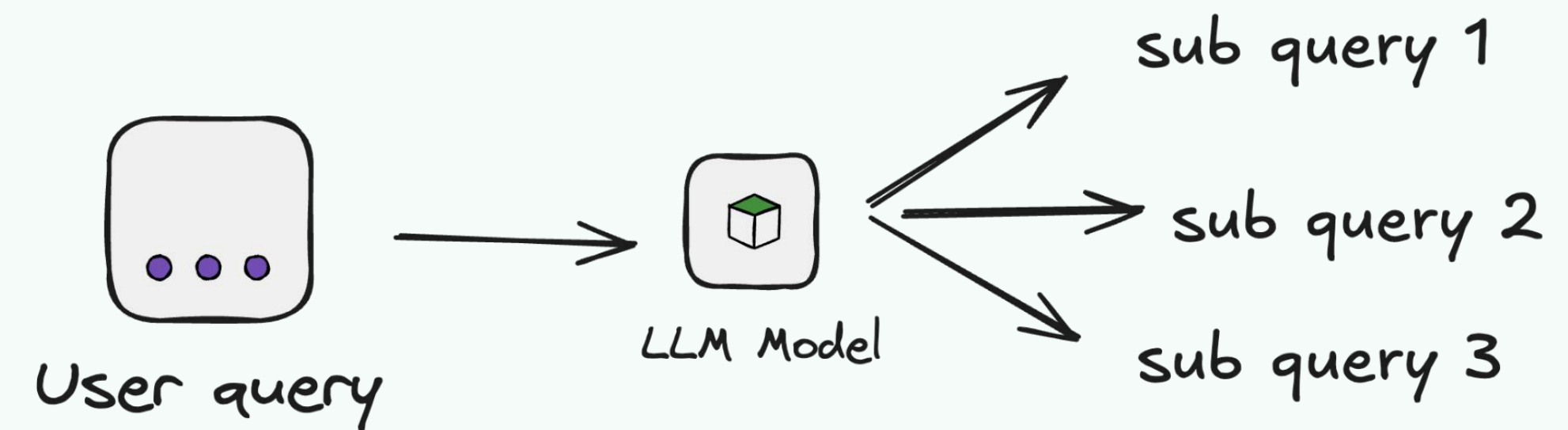


LLM to enhance the query to improve the retrieval

Method 1 -

Simple Query Enhancer Translated into a format that matches the retrieval mechanism.

Query Decomposition



LLM to divide the tasks to subqueries for improving the retrieval

Method 2 -

Query Decomposition Splitting a complex query into simpler, more manageable sub-queries.

5. Retrieval

Querying

1. Vector database

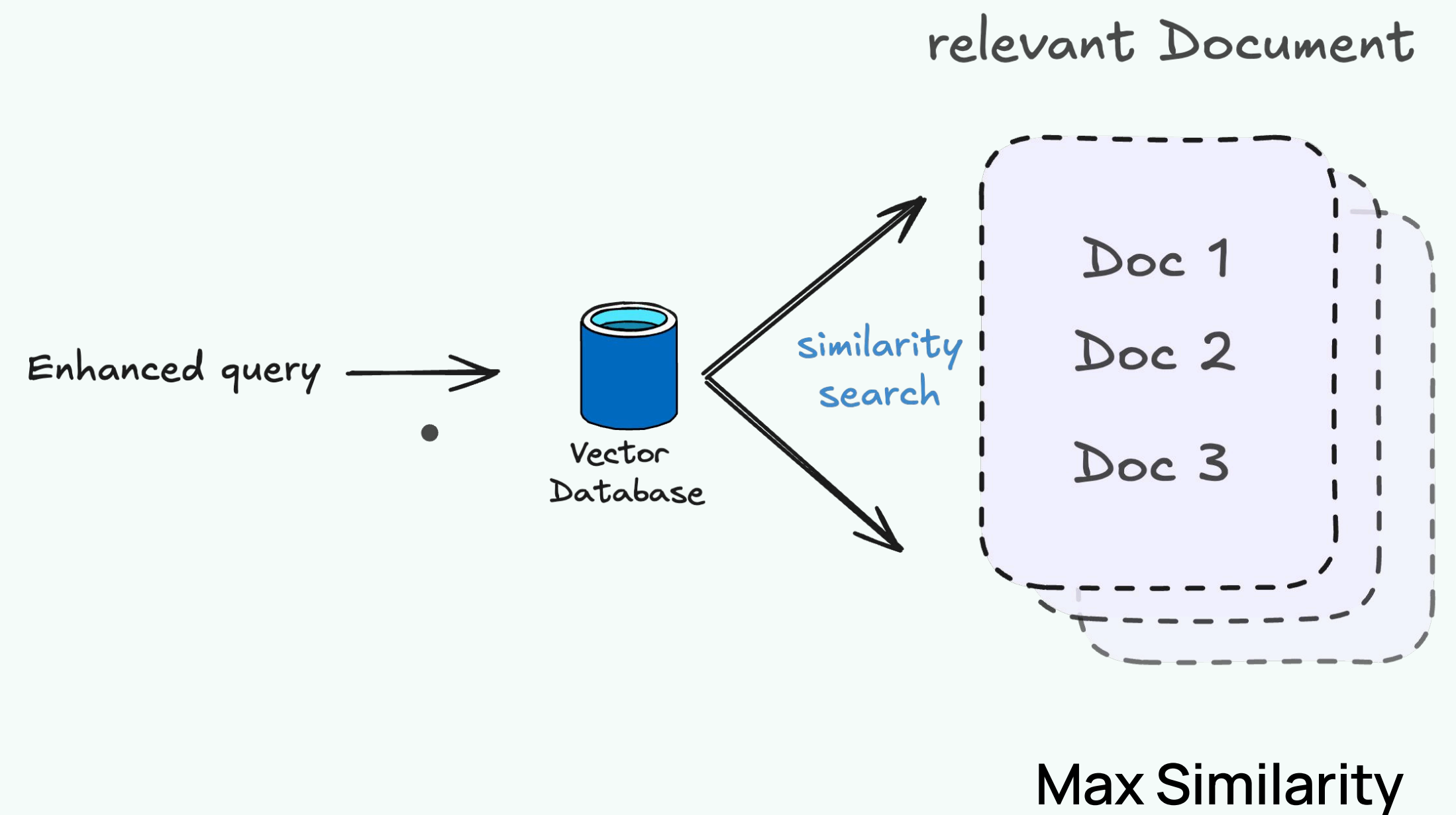
- retrieve the most relevant top k documents based on the semantic similarity to a query.

The current classification include,

- Infosys Financial Report

This module takes a user query and classifies it into a specific query type for a Vector DB. Currently, it supports the classification of financial report document. Based on the query, the module decides which type of document or information to retrieve.

ChromaDB retriver

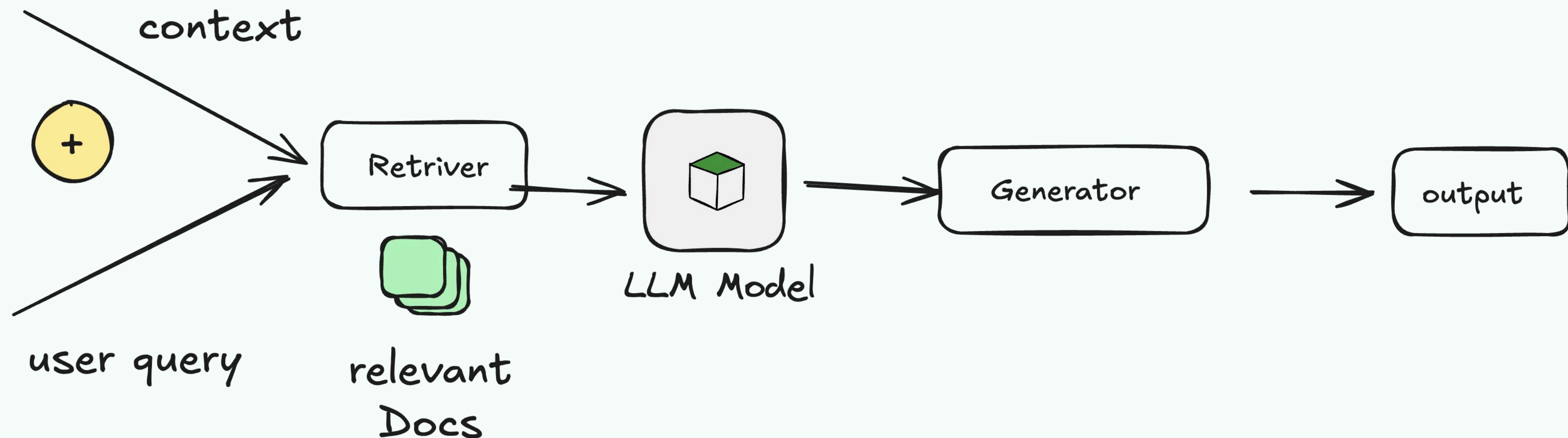


6. Generator

Generates answer based on input user query + context if from vector db or tooling output to the llm This module takes an user query, and makes a decision what type (Vector DB) of query the user requested,

The current classification include,

1. Upload any financial report
2. Infosys Financial report



7. Testing

Tested module with the P & I table data

Future Work

Enhancing table extraction

1. Continuous table pattern recognition -

- a. Repeat Headers for Continuity: For every chunk (including those from multi-page tables), repeat the column headers and table headers for continuity.

2. Handling Missing or Inconsistent Data

- a. Detecting Missing Data: detect the missing values and flag them to a None or N/A

3. Integrate with Multi modal Approach

Data

- 1. Index different types of data and structure
- 2. Data Cleaning
- 3. Caching
 - a. query cache vectordb

Future Work

RAG Pipeline

1. Query Translator

- a. Implementing various different methods in the query abstraction and query subtasks
- b. Implementing a entire pipeline to perform subtasks or subqueries
- c. Testing the performance of baseline user query vs enhanced queries
- d. Improving query decomposer

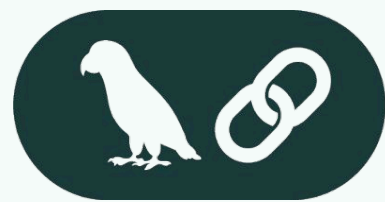
2. Query Retrival

- a. Trade-off analysis on the docs retrieved
- b. Improving the context
- c. Building relevancy module with the query and the docs
- d. Rerouting if the documents are not relevant
- e. Reranking, Rank fusion mechanism

3. Query Generator

- a. Improving the generation mechanism by testing accuracy - Human testing
- b. Using generation quality to query reconstruction or query improvement

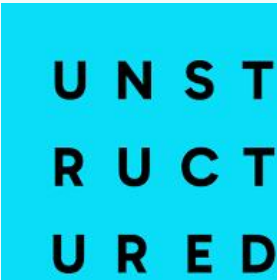
Technical



LangChain



OpenAI



Streamlit



docker



Chroma